MMDU MULLANA OFFICIAL SYLLABUS • BCSE-004 • UNIT 3

Unit 3: Data Types in Python (Lists, Tuples & Dictionaries)

Subject: Python Programming (BCSE-004) • **Exam Weightage:** 26 - 30 Marks • **Study Time:** 35 mins

🗺️ UNIT 3 SYLLABUS ROADMAP & TOPIC INDEX (TABLE OF CONTENTS)

2 Modules • 4 Topics • Pages 1–2

1. 1. Python Lists: Dynamic Arrays & Memory Mechanics Pg. 1

- TOPIC 1. Internal Implementation of Lists Pg. 1
- TOPIC 2. List Slicing & Method Complexities Pg. 1

2. 2. Tuples, Dictionaries & Hash Table Architecture Pg. 1–2

- TOPIC 1. Tuples vs Lists (12-Point Comparative Matrix) Pg. 1
- TOPIC 2. Dictionaries & Hash Table Architecture Pg. 2

1. Python Lists: Dynamic Arrays & Memory Mechanics

1. Internal Implementation of Lists

A Python **List** is implemented in CPython as a **contiguous array of 8-byte pointers** to heap objects. To achieve amortized $O(1)$ appends, Python uses a geometric over-allocation strategy: `allocated_slots = (newsize + (newsize >> 3) + 6) & ~3`.

2. List Slicing & Method Complexities

Method	Time Complexity	Description & In-place Mutation
`lst.append(x)`	$O(1)$ amortized	Appends element to end of array
`lst.pop()`	$O(1)$	Removes and returns last element
`lst.pop(0)` / `insert(0, x)`	$O(n)$	Forces n memory pointer shifts
`lst.remove(val)`	$O(n)$	Linear scan; removes first occurrence or raises `ValueError`
`lst.sort()`	$O(nlogn)$	In-place **Timsort** (hybrid merge + insertion sort)

```
# Shallow Copy vs Deep Copy
import copy
orig = [[1, 2], [3, 4]]
shallow = orig.copy()        # Inner sublists share identical references!
deep = copy.deepcopy(orig)   # Completely independent recursive duplication

orig[0][0] = 999
print(shallow[0][0])         # 999 (Mutated!)
print(deep[0][0])            # 1 (Safe & untouched!)
```

2. Tuples, Dictionaries & Hash Table Architecture

1. Tuples vs Lists (12-Point Comparative Matrix)

- **Immutability:** Tuples cannot be modified in-place; provides read-only data integrity.
- **Memory Optimization:** Sized precisely to element count with zero spare capacity overhead.
- **Hashability:** Tuples can serve as dictionary keys and set elements if all contained items are immutable.
- **Single-Element Trap:** `t = (5)` is an `int`. Must write `t = (5,)` with trailing comma!

2. Dictionaries & Hash Table Architecture

Dictionaries store associative `{key: value}` pairs. In modern Python (3.7+), dicts maintain **insertion order** via a two-array architecture: a sparse hash index table referencing a dense array of entries.

Key Property	Academic Rule & University Justification
1. Must be Hashable	Keys must implement `__hash__()` and `__eq__()`. Immutable types (`str`, `int`, `tuple`) are hashable. Mutable types (`list`, `dict`, `set`) raise `TypeError: unhashable type`.
2. Keys Must be Unique	Duplicate keys cannot exist; assigning to an existing key silently overwrites the previous value.

```
# Dictionary Comprehension & Safe Lookup
students = {"Bhavya": 98, "Aarav": 85, "Diya": 92}
distinction = {k: v for k, v in students.items() if v >= 90}
print(distinction)  # {'Bhavya': 98, 'Diya': 92}

# Safe lookup prevents KeyError crashes
score = students.get("Karan", 0)  # Returns 0 default instead of crashing!
```